OpenTech Connect 2026 · Hands-on Workshop

Hands-On with Docling for IBM watsonx

Convert, Extract, and Build Your Own Document-Powered App

Cesar Berrospi Ramis
Senior Researcher, IBM Software
ceb@zurich.ibm.com

WORKSHOP OVERVIEW
This workshop guides you through automating enterprise document intelligence using **Docling for IBM watsonx**. You will process heterogeneous compliance documents (PDF, DOCX, EPUB), export to the ISO-standard **DocLang** format, and choose between two tracks: **Track A (IBM Bob AI Assistant — No Code)** or **Track B (Python SDK & Notebook — Developer)**.

Tool / Service	Access & Free Trial URL	Role
IBM Bob	https://bob.ibm.com/download	AI assistant & development workspace
Docling for watsonx	https://www.ibm.com/products/docling	Managed document conversion service
IBM watsonx.ai	https://www.ibm.com/products/watsonx-ai	Foundation models & embeddings (Track B)
DocLang Viewer	https://doclang.ai/viewer/	Browser viewer for .dclx archives

1. Introduction & Scenario

You are a compliance analyst at **Vantara Bank**, a mid-sized financial institution operating across multiple jurisdictions. Every quarter, you receive a bundle of regulatory submissions, audit reports, and risk disclosures — arriving in various formats (PDF, DOCX, EPUB). Your task is to extract key risk metrics and regulatory obligations to populate a standardized **Compliance Assessment Form**.

In this workshop, you will use **Docling for IBM watsonx** to automate this end-to-end pipeline without needing local GPUs.

File	Format	Target Fields Extracted
vantara_risk_disclosure.pdf	PDF (4 pages)	Entity name, reporting period, total risk exposure USD, highest risk category, regulator reference #, charts & tables
q3_audit_summary.docx	DOCX (6 sections)	Material findings (list), sign-off date, cross-verification of risk exposure & reference #
regulatory_guidelines_2026.epub	EPUB (7 chapters)	Key regulatory obligations (list), AML requirements, reporting deadlines
compliance_assessment_form.json	JSON Schema	Standardized 8-field assessment target filled at the end of both tracks

2. Getting Started (Setup & Bootstrap)

Run the workshop from your personal machine or workstation. The repository provides an automated bootstrap.sh script that verifies prerequisites, installs uv, creates an isolated Python 3.12 virtual environment, installs the required SDKs (docling-client, dclq), and initializes your configuration.

Step 1 — Clone the Repository & Navigate to the Workshop

```
git clone https://github.com/docling-project/docling-workshops.git
cd docling-workshops/workshops/2026_11_12
```

Step 2 — Run the Bootstrap Script

```
./bootstrap.sh
```

Step 3 — Configure Your Credentials in .env

The bootstrap copies .env.example to .env. Open .env in your editor and enter your Docling service credentials (obtained in Part 1) and optional watsonx credentials for Track B:

```
DOCLING_SERVICE_URL=https://
DOCLING_SERVICE_API_KEY=

# watsonx.ai credentials (Track B)
WX_API_KEY=
WX_PROJECT_ID=
WX_URL=https://us-south.ml.cloud.ibm.com
```

Step 4 — Verify Connection & Setup

Run python scripts/test_connection.py to verify that your environment is properly set up. This script automatically checks that:

- docling-client is installed in the active virtual environment
- .env configuration contains valid credentials (not placeholder defaults)
- An end-to-end conversion test against the Docling service executes successfully

```
source .venv/bin/activate
python scripts/test_connection.py
```

3. Part 1 — Get Ready with Docling for IBM watsonx (20 min)

In this section, you will use the no-code web UI to explore managed conversion and inspect DocLang outputs.

3.1 Start Your Free Trial

1. Navigate to **https://www.ibm.com/products/docling** and request the free trial.
2. Sign in with your IBMid (or create a free account).
3. Once provisioned, click *Access your trial now* to open the **Docling for IBM watsonx** dashboard.

3.2 Explore the Workbench

1. In the Workbench, click **Upload a document** and select docs/vantara_risk_disclosure.pdf.
2. In the **Tasks** view, inspect the extracted hierarchy: structural nodes (sections, paragraphs, tables) on the left and rendered visual preview on the right.
3. Click table nodes to view reconstructed rows, column headers, and multi-line cells.
4. Try a custom conversion: click **Create task** → **Single**, re-upload the PDF, check **DCLX** output format, and enable **Picture classification**.
5. Under **Actions**, select **Download** to save the conversion archive (.dclx).

3.3 Inspect DocLang in the DocLang Viewer

DocLang is an open ISO standard for structured document representation (**https://doclang.ai**).

1. Open **https://doclang.ai/viewer/** in your browser.
2. Drag and drop your downloaded .dclx file.
3. In the **Layers** menu, uncheck *Furniture* to filter out headers and footers from semantic content.

3.4 Get Service URL and API Key

1. In the Docling dashboard, navigate to **API examples** to locate your **Service URL**.
2. Click **Get API key** → **Generate an API key**, copy the key, and paste both values into your .env file.

4. Part 2 — Choose Your Track (50 min)

Both tracks process all three documents and populate the identical 8 fields of the Compliance Assessment Form. Choose the track that fits your role:

Feature	Track A: IBM Bob (No-Code)	Track B: Python SDK (Developer)
Target Audience	Analysts, consultants, no coding required	Developers, data scientists, engineers
Driver	Natural language conversation with IBM Bob assistant	Interactive Jupyter notebook (docling_lab.ipynb)
Output Deliverable	JSON form, executive summary, Markdown table	Pydantic typed object, HTML report, DocLang file
Skill / Library	Built-in docling-lab Bob skill	docling-client, dclq, watsonx.ai

Track A — Document Intelligence with IBM Bob (No Code)

1. Launch and Log into IBM Bob: Download Bob from **https://bob.ibm.com/download** and sign in with your IBMid.
2. Load the Lab Skill: Bob automatically detects the skill in .bob/skills/docling-lab/. When prompted on first use, click *Approve skill tools for task*.
3. Converse with Bob: Use the following prompts in the chat panel to convert documents and extract data:

> ***Prompt:*** *Convert docs/vantara_risk_disclosure.pdf and return the document key.*

> ***Prompt:*** *Show me the heading outline and extract the main risk categories described in the document.*

> ***Prompt:*** *Convert all three documents in docs/ (vantara_risk_disclosure.pdf, q3_audit_summary.docx, regulatory_guidelines_2026.epub). Extract the eight fields for compliance_assessment_form.json and return as structured JSON.*

> ***Prompt:*** *Write a 3-sentence compliance executive summary highlighting total risk exposure, highest risk category, and key obligations. Then build a DoclingDocument table and export to Markdown and DocLang (.dclx).*

Track B — Document Intelligence with Python (Developer)

1. Open the Notebook: Open docs/docling_lab.ipynb in IBM Bob, VS Code, or JupyterLab.
2. Select Kernel: Ensure the Python kernel points to workshops/2026_11_12/.venv.
3. Work Through the 9 Notebook Sections:

Section	Topic	Core API & Actions
1	Connect to Service	DoclingServiceClient(url, api_key)
2	Multi-doc Conversion	client.convert_all([pdf, docx, epub])
3	Structure Inspection	Iterate document nodes, tables, text items
4	DocLang & dclq CLI	Export .dclx and query with dclq outline / inspect
5	Hybrid Chunking	Structure-aware chunking sized for embedding models
6	RAG with watsonx.ai	watsonx embeddings + vector store + grounded answers
7	Structured Extraction	Extract into Pydantic ComplianceForm schema
8	Report Generation	Generate executive summary & HTML compliance report
9	(Optional) Docling Agent	Automated document editing with DoclingEditingAgent

5. Compare the Two Routes & Key Takeaways

Both routes extract the same eight fields from three distinct formats. Compare the approaches:

Dimension	Track A (IBM Bob)	Track B (Python SDK)
Schema Definition	Specified via conversational prompt	Strongly-typed Pydantic model
Inspection & Trace	Tool execution trace in chat UI	Cell-by-cell execution output & raw JSON
Ideal Use Case	Rapid ad-hoc exploration of unfamiliar document sets	Automated, reproducible enterprise batch pipelines

Two Critical Observations to Check:

• **Cross-Document Consistency:** Notice that total_risk_exposure_usd and regulator_reference_number appear in both the PDF and DOCX. Verify whether both sources report matching numbers ($4.82B USD).
• **Structural Integrity:** Observe that tables with merged headers and multi-column figures are preserved as semantic tables rather than flattened text, preventing loss of cell relationships.

6. Quick Reference & CLI Commands

Python Service Client Snippet:

```
from docling.service_client import DoclingServiceClient

with DoclingServiceClient(url=SERVICE_URL, api_key=API_KEY) as client:
    result = client.convert(source='docs/vantara_risk_disclosure.pdf')
    print(result.document.export_to_markdown())
```

DocLang CLI (dclq) Commands:

```
dclq inspect output/vantara_risk_disclosure.dclx        # Structural inventory
dclq outline output/vantara_risk_disclosure.dclx        # Heading tree & XPaths
dclq grep -i 'risk exposure' output/*.dclx              # Semantic pattern search
```

Resource Links:

• **Docling Open Source:** https://github.com/docling-project/docling
• **Docling for IBM watsonx:** https://www.ibm.com/products/docling
• **IBM Bob Assistant:** https://bob.ibm.com/download
• **IBM watsonx.ai:** https://www.ibm.com/products/watsonx-ai
• **DocLang Viewer:** https://doclang.ai/viewer/
• **Docling MCP Server:** https://github.com/docling-project/docling-mcp